

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks: 25

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I V.Madhulika (192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Answer:

Possible Causes:

- Learning rate too high, causing weight updates to overshoot and oscillate around the separating boundary instead of settling.
- Poor weight/bias initialization placing the decision boundary far from a workable region.
- Bias term omitted or not updated, preventing the boundary from shifting to correctly separate the classes.
- Bug in the update rule (e.g., wrong sign, missing input term, or updating with the wrong error value).

Debugging Strategies:

- Log the weight vector, bias, and misclassification count after every epoch to check whether error is decreasing or oscillating.
- Test the perceptron on a tiny, hand-verifiable dataset (e.g., AND/OR gate) to confirm the update rule is implemented correctly.
- Print net input and predicted output for a few samples and manually verify the calculation matches the perceptron update formula.

Optimization Techniques:

- Use a smaller, appropriately tuned learning rate, optionally with a decay schedule.
- Initialize weights to small random values (not all zero) and always include and update the bias.
- Shuffle training samples each epoch to avoid cyclic update patterns that stall convergence.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

Answer:

Potential Reasons:

- Vanishing gradients caused by sigmoid/tanh activations in deep networks, where gradients shrink as they are propagated backward through many layers.
- Poor weight initialization (e.g., all zeros or too large) causing symmetric or saturated neurons.
- A fixed, suboptimal learning rate that is too small (slow progress) or too large (unstable jumps between local minima).

Debugging Steps:

- Plot the training loss curve over epochs — a nearly flat curve suggests vanishing gradients or too small a learning rate.
- Inspect gradient magnitudes layer by layer; gradients that shrink drastically in earlier layers confirm the vanishing gradient problem.
- Try training with a much smaller subnetwork or fewer layers to isolate whether depth is the cause.

Optimization Techniques:

- Replace sigmoid/tanh with ReLU or its variants (Leaky ReLU) to mitigate vanishing gradients.
- Use proper weight initialization schemes such as Xavier/He initialization.
- Apply batch normalization to stabilize activations, and use adaptive optimizers such as Adam or RMSProp instead of plain gradient descent to automatically adjust the effective learning rate.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

Answer:

Possible Issues:

- Chromosome encoding that does not faithfully represent all valid hypotheses, restricting the search space unnecessarily.
- A fitness function that is poorly correlated with actual solution quality, misleading selection.
- Premature convergence, where the population loses diversity too early and clusters around a local optimum.
- Mutation rate too low to escape local optima, or too high, disrupting good solutions before they can be exploited.

Systematic Debugging:

- Track population diversity (e.g., average Hamming distance between chromosomes) across generations; a sharp early drop signals premature convergence.
- Log the best and average fitness per generation across multiple runs to see if results vary widely (indicating instability) or are all mediocre (indicating a fitness function or encoding flaw).
- Manually verify the fitness function on a few known-good and known-bad hypotheses to confirm it ranks them correctly.

Optimization Strategies:

- Tune the mutation rate (typically small, e.g., 1–5%) to balance exploration and exploitation.
- Adjust selection pressure (e.g., tournament size) so that fitter individuals are favored without eliminating diversity too quickly.
- Apply elitism — carrying the best individual(s) unchanged into the next generation — to guarantee solution quality never regresses across generations.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify

debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

Answer:

Debugging Steps:

- Compare training vs validation loss curves; a widening gap confirms overfitting driven by excess model capacity relative to available data.
- Count model parameters relative to the size of the training set to check for over-parameterization.
- Profile training time per generation/epoch to identify which components (deep network training or genetic-programming evaluation) dominate computational cost.

Optimization Techniques:

- Regularization (L1/L2) to penalize unnecessarily large weights and simplify the learned function.
- Dropout to reduce co-adaptation between neurons and improve generalization.
- Pruning – removing redundant weights, neurons, or genetic-programming branches that contribute little to performance, reducing both overfitting and computational load.
- Use a smaller/efficient surrogate model or reduced-fidelity evaluation during genetic programming's fitness evaluation step to cut computation cost, reserving full evaluation for the most promising candidates.
- Increase effective training data via augmentation or gather more samples to better match model capacity.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Answer:

Possible Causes:

- Overfitting due to the tree growing too deep and modeling noise/outliers in the training data as if they were genuine patterns.
- Presence of noisy or mislabeled samples that the tree tries to perfectly classify.
- Irrelevant or redundant features adding spurious splits that do not generalize.

Debugging Steps:

- Compare training accuracy to test/validation accuracy — a very large gap confirms overfitting.
- Inspect tree depth and the number of leaf nodes; excessively deep trees with very few samples per leaf are a strong warning sign.
- Use k-fold cross-validation to check whether performance is consistent across different data splits.

Optimization Methods:

- Pruning (pre-pruning via max depth/minimum samples per leaf, or post-pruning via cost-complexity pruning) to remove branches that do not improve validation performance.
- Cross-validation to select the optimal tree complexity/hyperparameters objectively.
- Feature selection to remove irrelevant/noisy attributes before training, reducing spurious splits.

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Answer:

Potential Issues:

- Overly aggressive or inconsistent data augmentation distorting images beyond realistic variation, confusing the model.
- Vanishing gradients in deeper convolutional stacks limiting effective learning in early layers.
- Incorrectly configured batch normalization (e.g., wrong axis, or running statistics not updated properly), causing inconsistent behavior between training and inference.
- Learning rate too high, causing loss to fluctuate wildly between epochs.

Debugging Strategies:

- Plot training/validation accuracy and loss across epochs to see if instability correlates with specific augmentation settings or a high learning rate.
- Temporarily disable data augmentation and batch normalization one at a time to isolate which component causes instability.
- Visualize a batch of augmented images to confirm augmentation is realistic and not overly destructive.

Optimization Techniques:

- Use moderate, well-validated augmentation (rotation, flip, slight color jitter) rather than extreme transformations.
- Use ReLU-family activations with proper weight initialization to reduce vanishing gradients.
- Correctly configure batch normalization and verify running mean/variance are properly tracked; reduce the learning rate or use a learning-rate scheduler/warm-up to stabilize training.

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters.

Answer:

Possible Causes:

- Sparse or poorly designed reward signal that gives the agent little useful feedback about progress toward the goal.
- Insufficient exploration, causing the agent to repeatedly exploit an early, sub-optimal strategy without discovering better ones.
- An incomplete or uninformative state representation that hides information the agent needs to make good decisions.

Debugging Methods:

- Plot cumulative reward per episode; a flat or highly noisy curve suggests reward sparsity or an exploration problem.
- Log the distribution of actions taken over training — heavy concentration on very few actions early on indicates insufficient exploration.

- Manually inspect whether the state representation contains all information a human would need to act optimally in that environment.

Recommended Improvements:

- Reward shaping – provide intermediate rewards for partial progress toward the goal instead of only a sparse terminal reward.
- Exploration strategies – use epsilon-greedy with a decaying epsilon, or entropy bonuses, to ensure sufficient exploration before converging.
- Tune hyperparameters such as the discount factor, learning rate, and batch size, and enrich the state representation with additional relevant features.

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

Answer:

Possible Reasons:

- Random centroid initialization causing convergence to different local optima on different runs.
- An incorrect choice of K that does not match the natural number of groups in the data.
- Outliers pulling centroids away from the true cluster centers, distorting assignments.
- Unscaled features, causing attributes with larger numeric ranges to dominate the distance calculation.

Debugging Approaches:

- Run K-means multiple times with different random seeds and compare resulting inertia/cluster assignments to check sensitivity to initialization.
- Use the Elbow method or silhouette score across a range of K values to check whether the chosen K is appropriate.
- Visualize the data (or a 2D projection via PCA) to check for obvious outliers or non-spherical cluster shapes that violate K-means' assumptions.

Optimization Strategies:

- Use K-means++ initialization to select well-spread initial centroids, improving consistency across runs.
- Normalize/standardize features before clustering so no single feature dominates the distance metric.
- Use silhouette analysis to select the optimal number of clusters and consider outlier removal or robust clustering variants if outliers are influential.

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

Answer:

Possible Issues:

- Using a linear kernel on data that has a genuinely nonlinear decision boundary.

- Poorly tuned hyperparameters (C , γ) leading to either an overly rigid or overly flexible decision boundary.
- Unscaled features causing the distance-based kernel computations to be dominated by large-magnitude attributes.

Debugging Techniques:

- Visualize the data (2D/PCA projection) to check whether classes are linearly separable or require a nonlinear boundary.
- Compare training vs test accuracy for different kernels (linear vs RBF vs polynomial) to identify underfitting or overfitting.
- Check the range/scale of each feature before training to detect scaling issues.

Optimization Methods:

- Switch to a nonlinear kernel such as RBF (Gaussian) for data with complex boundaries.
- Use grid search or randomized search with cross-validation to jointly tune C (regularization strength) and γ (kernel width).
- Standardize/normalize all features before training so the kernel's distance calculations are meaningful across attributes.

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

Answer:

Causes:

- Vanishing gradients during backpropagation-through-time, which prevents the network from learning relationships between tokens that are far apart in the sequence.
- Limited memory capacity of a simple RNN's hidden state, which gets overwritten as the sequence grows longer.

Debugging Steps:

- Test the model on sequences of increasing length and measure how accuracy degrades — a sharp drop with longer sequences confirms long-term dependency failure.
- Inspect gradient magnitudes for early time steps during backpropagation-through-time; near-zero gradients confirm vanishing gradients.
- Construct a synthetic task requiring recall of information from many steps back to directly probe memory capacity.

Proposed Solutions:

- Replace vanilla RNN cells with LSTM or GRU units, whose gating mechanisms preserve gradient flow and retain information over longer sequences.
- Add an attention mechanism so the model can directly attend to relevant earlier tokens rather than relying solely on the compressed hidden state.
- Use gradient clipping and improved training strategies (e.g., truncated backpropagation-through-time with adequate window size, curriculum learning on shorter sequences first) to stabilize and enhance training.

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance.

Answer:

Possible Causes:

- Low accuracy on both training and test sets indicates underfitting, often from a kernel that is too simple (e.g., linear) for the data's true structure.
- A poorly chosen C (too small, allowing too many margin violations) or gamma (too small, making the RBF kernel too 'flat' to capture local structure) can severely limit the model's expressive power.
- Unscaled features distorting the kernel's distance/similarity computations.

Systematic Debugging Steps:

- Since both training and test accuracy are low, first rule out data issues: check for mislabeled samples or incorrect target encoding.
- Visualize the feature space to assess whether classes are linearly separable or require a nonlinear kernel.
- Try a baseline default configuration (RBF kernel, default C and gamma) first, to isolate whether the current poor performance stems from hyperparameters rather than the algorithm choice itself.

Optimization Techniques:

- Kernel tuning – switch to RBF or polynomial kernels if the linear kernel underfits.
- Feature scaling/normalization – standardize all features so the kernel computation is not skewed by attributes with larger numeric ranges.
- Grid search (or randomized search) with cross-validation over C and gamma to systematically find the hyperparameter combination that best fits the data.

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques.

Answer:

Possible Issues:

- An incorrect number of clusters (K) forcing the algorithm to either merge genuinely distinct groups or artificially split a single natural cluster.
- Poor random initialization placing initial centroids close together, causing K-means to converge with some centroids capturing very few points.
- Outliers or extreme values pulling one or more centroids away from dense regions, creating imbalanced, unrepresentative clusters.

Debugging Approaches:

- Visualize cluster sizes and centroid locations to spot obvious imbalance or centroids stranded near outliers.

- Re-run clustering with multiple random seeds and compare cluster-size distributions to check sensitivity to initialization.
- Examine the data for extreme outliers using box plots or z-scores before clustering.

Improvements:

- Use the Elbow method (or silhouette score) to choose a more appropriate value of K that matches the data's natural structure.
- Use K-Means++ initialization to spread initial centroids and reduce the chance of poor local optima.
- Detect and remove (or separately handle) outliers before clustering so they do not distort centroid placement.

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization.

Answer:

Possible Causes:

- Vanishing gradients across many time steps, preventing the network from learning dependencies far back in the sequence.
- Training sequence length too short to expose the model to the long-range patterns it needs to learn.
- An architecture with too few hidden units or an unsuitable cell type (plain RNN instead of gated units) for the complexity of the time-series.

Debugging Steps:

- Evaluate prediction accuracy as a function of how far back the relevant signal occurs, to directly test long-term dependency capture.
- Monitor gradient norms during backpropagation-through-time to detect vanishing (or exploding) gradients.
- Check if input sequences are normalized/scaled consistently, since inconsistent scale across time-series segments can cause unstable outputs.

Optimization Techniques:

- Replace plain RNN cells with LSTM or GRU units to better preserve information over long sequences.
- Apply gradient clipping to prevent exploding gradients and stabilize training.
- Normalize input sequences (e.g., min-max or z-score normalization per feature) and increase training sequence length where feasible so the model is exposed to longer-range patterns.

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods.

Answer:

Possible Issues:

- Strong correlations between words/features violate Naïve Bayes' conditional independence assumption, distorting probability estimates.
- Insufficient text preprocessing (no stop-word removal, no stemming/lemmatization) leaving noisy, uninformative tokens in the feature set.
- Class imbalance in the training data biasing the classifier's prior probabilities toward the majority class.

Debugging Strategies:

- Inspect the most informative features/words per class; if many high-weight features are clearly irrelevant (e.g., very common stop words), preprocessing is likely inadequate.
- Check per-class prediction accuracy separately — much lower accuracy on the minority class signals dataset imbalance.
- Test performance with and without removing highly correlated feature groups to gauge how much the independence violation is hurting predictions.

Suggested Improvements:

- Feature selection – remove stop words, apply stemming/lemmatization, and use techniques like chi-square or mutual information to keep only the most discriminative words.
- Smoothing techniques – apply Laplace (add-one) smoothing to handle unseen words and avoid zero probabilities.
- Data balancing methods – use oversampling of the minority class, undersampling of the majority class, or class-weighted priors to correct for imbalance.

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance.

Answer:

Possible Causes:

- Sparse rewards (e.g., only given upon reaching the goal) provide little guidance during most of the navigation, making learning slow and unstable.
- An improper exploration-exploitation balance — too much exploitation early leads to a poor fixed policy, while too much exploration prevents the agent from ever settling on a good one.
- A reward function that inadvertently encourages unintended behavior (reward hacking), such as rewarding movement without penalizing collisions or wasted steps.

Debugging Approaches:

- Plot episode reward and success rate over training; persistent erratic swings without an upward trend point to an unstable exploration strategy or reward design flaw.
- Visualize agent trajectories in the environment to check for repetitive, non-productive movement patterns indicating exploitation of a reward loophole.
- Test with a simplified version of the environment (e.g., smaller grid) to see if the agent can learn a sensible policy at all, isolating whether the issue is scale-related or fundamental to reward/exploration design.

Optimization Strategies:

- Reward shaping – add small intermediate rewards (e.g., for reducing distance to the goal) to provide denser learning signal without changing the optimal policy.
- Epsilon decay tuning – start with high exploration (high epsilon) and gradually decay it as the agent gains experience, ensuring a good balance between exploring the environment and exploiting learned knowledge.
- Policy optimization methods – use more stable algorithms such as Proximal Policy Optimization (PPO) or Actor-Critic methods, which constrain policy updates and tend to produce smoother, more reliable learning than naive value-based methods in complex navigation tasks.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand